Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo (3–5 linhas)

Implementar, em C, as funções de sincronização do sistema de embarque de passageiros do metrô descrito no enunciado: station_init, station_wait_for_car, station_on_board e station_load_car, utilizando exclusivamente pthread_mutex e pthread_cond_t (sem semáforos, sem busy-wait, com um único lock por station). A entrega inclui o código-fonte (sync_car.c/sync_car.h), o Makefile de compilação, este relatório e o evidencias.log do processo de desenvolvimento.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Requisito do enunciado	Status	Como testar / evidência
Implementado em C, executável em Linux/Unix/macOS	OK	make && ./sync_car
Usa pthread_mutex_init/lock/unlock, pthread_cond_init/signal/wait/broadcast	OK	Ver sync_car.c
Não usa semáforos ou outras primitivas	OK	Ver sync_car.c (apenas mutex + cond)
Não usa mais que um único lock por struct station	OK	struct station em sync_car.h possui um único pthread_mutex_t lock
Assume no máximo um carro por vez na estação	OK	car_present controla isso; teste do professor respeita essa premissa
Permite embarque simultâneo de passageiros	OK	station_wait_for_car não serializa passageiros entre si
Sem busy-wait	OK	Toda espera usa pthread_cond_wait dentro de while

Compilação via Makefile, gerando executável sync_car	OK	make gera sync_car

3. Como Reproduzir

Comandos reais, copiáveis. Se não rodar com o que está aqui, não é avaliado.

Compilar:

```
make clean
make
```

Executar:

```
./sync_car
```

4. Arquitetura (resumida)

Arquivos e responsabilidades:

1. **sync_car.h → declara struct station e as assinaturas das 4 funções.**
2. **sync_car.c → implementa a lógica de sincronização.**
3. **sync_car-runner.c → arquivo de teste fornecido pelo professor (não alterado).**
4. **Makefile → compilação (all) e limpeza (clean).**

Decisões técnicas:

1. **Um único mutex + uma única condition variable compartilhada → Motivo: atende ao requisito de um único lock por station e simplifica o raciocínio, já que todo pthread_cond_wait já está protegido por while checando a condição real → Consequência: alguns wakeups são "desnecessários" (broadcast acorda threads que não se tornam elegíveis), mas isso não compromete corretude, só performance marginal — irrelevante pra esse projeto.**
2. **Contadores explícitos (waiting, to_board) em vez de apenas free_seats → Motivo: a regra de parada de station_load_car ("cheio OU todos que esperavam embarcaram") exige saber quantos passageiros ainda estão na fila e quantos já reservaram assento mas não confirmaram → Consequência: struct fica com mais campos, mas a lógica de parada fica direta e legível.**
3. **station_wait_for_car reserva o assento (free_seats--) antes de retornar → Motivo: garante atomicidade da reserva sob lock, evitando que dois passageiros disputem o mesmo assento → Consequência: a contagem de assentos livres é sempre consistente mesmo com múltiplos passageiros concorrendo.**

5. Estratégias e Diário de Desenvolvimento

Isto é o núcleo do relatório.

5.1 Estratégias da semana

Uma linha por abordagem diferente que você tentou. Se você não mudou de abordagem, preencha só S1.

Estratégia	**Nome curto**	**Contexto**	**Motivo da troca (se houve)**

S1	Mutex único + cond var única + contadores (car_present, free_seats, waiting, to_board)	Definida antes de começar a codar, a partir da leitura do enunciado e das restrições (1 lock, sem semáforo, sem busy-wait)	Não houve troca de abordagem — a estratégia inicial se manteve válida do início ao fim, inclusive nos casos-limite testados.

5.2 Diário de Tentativas

Uma linha por tentativa relevante, marcando a qual estratégia (S1/S2/S3) ela pertence, obviamente precisa ser em ordem cronológica. A coluna "Quando" precisa bater com a ordem cronológica do evidencias.log, é isso que comprova que o fluxo é real. "Hipótese/Causa" só é preenchida quando o resultado foi falha ou parcial.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	0
1	S1	Compilação inicial dos stubs de sync_car.c junto com sync_car-runner.c via make	OK	—	Mon Nov 2 19:52:34 -03 2026	saída do gcc com warning em alarm_handler (parâmetro não usado, arquivo do professor)
2	S1	Implementação de station_init (mutex + cond var + contadores a zero) e atualização da struct em sync_car.h	OK	—	Mon Nov 2 20:11:00 -03 2026	compilação limpa, só o warning conhecido do arquivo do professor
3	S1	Teste unitário próprio para station_wait_for_car: passageiro bloqueia até carro chegar com assento livre	OK	—	Tue Nov 3 23:12:36 -03 2026	bloqueio confirmado por 1s, depois retorno com free_seats=0, waiting=0, to_board=1
4	S1	Implementação de station_load_car e station_on_board; primeira execução completa de ./sync_car	OK	—	Tue Nov 3 23:31:15 -03 2026	3 carros (31/23/46 passageiros), total 100/100, “Looks good!”

5	S1	Execução única de ./sync_car após implementar as 4 funções	OK	—	Tue Nov 3 23:43:42 -03 2026	3 carros, 31+23+46=100, “Looks good!”
6	S1	30 execuções seguidas de ./sync_car em loop, checando código de saída	OK	—	Tue Nov 3 23:46:31 -03 2026	nenhuma falha nas 30 rodadas
7	S1	20 execuções com timeout 5 para detectar travamentos	OK	—	Tue Nov 3 23:49:26 -03 2026	nenhuma falha/travamento
8	S1	Caso-limite: 1 único passageiro, 1 único assento	OK	—	Wed Nov 4 00:38:22 -03 2026	station_load_car só retornou após embarque e confirmação; free_seats=0 waiting=0 to_board=0
9	S1	Caso-limite: passageiro atrasado não embarca no 1º carro (já partido), embarca no 2º	OK	—	Wed Nov 4 00:45:32 -03 2026	1º carro: 2/2 embarcados, atrasado de fora; 2º carro: atrasado embarcou; fim com todos contadores zerados
10	S1	Caso-limite: station_load_car(0) com 2 passageiros já esperando	Parcial	Condição de corrida no próprio harness de teste: a main chamava station_load_car(0) logo após criar as 2 threads de passageiro, sem garantir que ambas já tivessem incrementado waiting. Como free_seats=0 faz a função retornar sem nenhum cond_wait (sem ceder o processador), é provável que só 1	Wed Nov 4 01:16:08 -03 2026	waiting=1 em vez do esperado 2; não é bug em sync_car.c, é falta de sincronização no próprio teste

				das 2 threads tivesse rodado até waiting++ até esse ponto. O resultado final permaneceu correto (ninguém perdido ou duplicado), mas o valor intermediário de waiting impresso ficou não-determinístico.		
11	S1	Correção: polling determinístico aguardando waiting==2 antes de chamar station_load_car(0)	OK	—	Wed Nov 4 01:29:38 -03 2026	waiting=2 antes e depois do station_load_car(0), confirmando que era o teste, não o sync_car.c, que precisava de sincronização
12	S1	Rodada final de validação: 40 execuções de ./sync_car com timeout 5	OK	—	Wed Nov 4 19:40:45 -03 2026	nenhuma falha/travamento em 40 rodadas

Regra prática: se algo deu muito errado (bug que te travou por horas), a linha da tabela resume em 1 frase e aponta para o trecho do evidencias.log com o detalhe.

6. Evidências

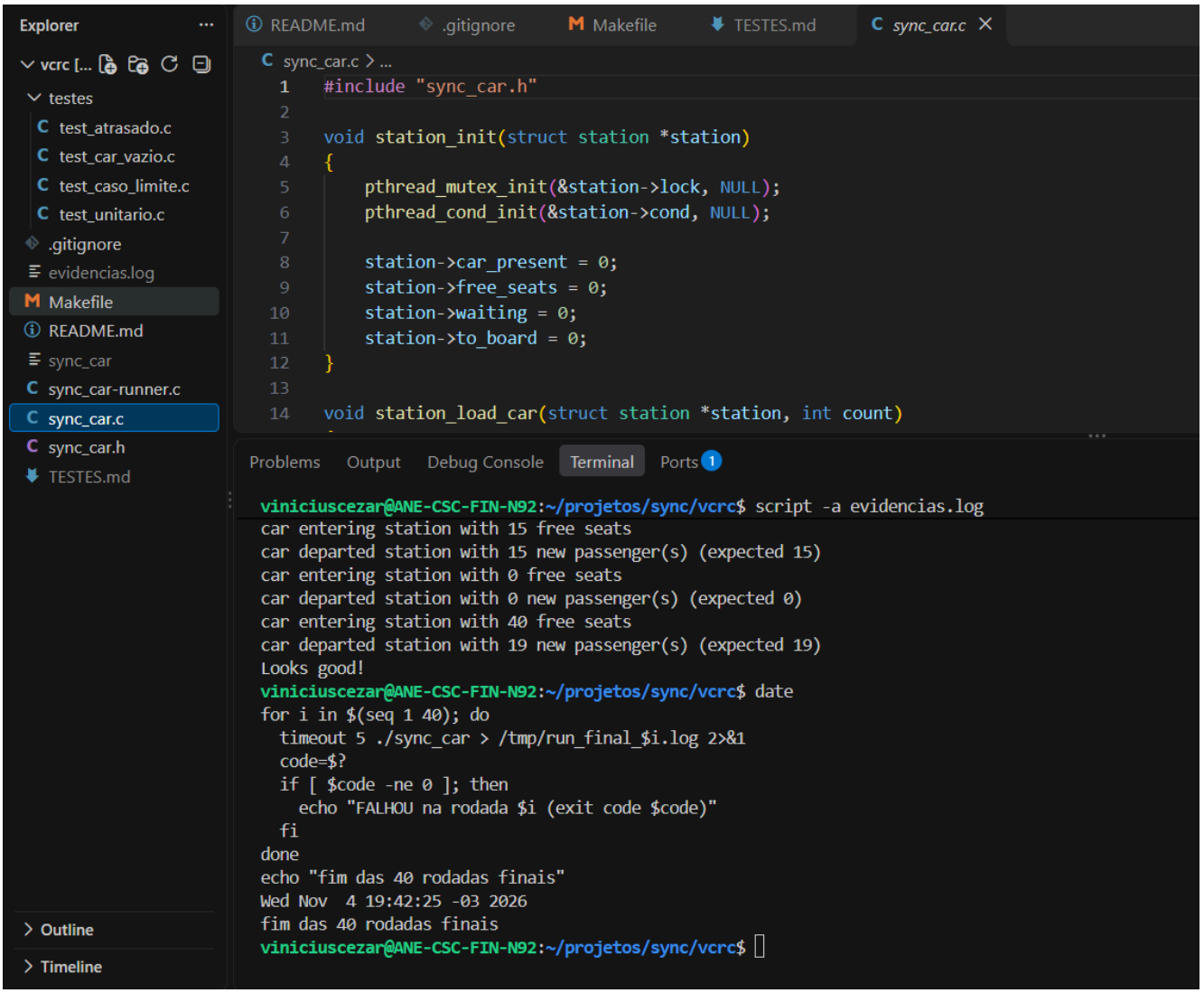
A regra continua: toda afirmação relevante precisa de prova. O que muda é como capturar essa prova, para não depender de print manual toda hora.

6.1 Log automático (faz o trabalho pesado por você)

Toda a sessão de trabalho foi registrada via script -a evidencias.log, iniciado com date, whoami e pwd no começo de cada sessão (o trabalho foi dividido em múltiplas sessões ao longo de alguns dias, cada uma reiniciando o log em modo append). O arquivo completo (evidencias.log) é entregue dentro do .tar.

6.2 Prints

• Validação final: teste passando / saída correta.



Prints extras só se algo excepcional aconteceu e o log sozinho não explica, você precisa ter esse feeling, mas posso dar uma ajuda (ex.: comportamento visual, crash com stack trace longo).

7. Uso de IA

Obrigatório preencher mesmo se a resposta for "não utilizei".

Onde usei IA (2–3 linhas): utilizei assistentes de IAcomo ferramentas de apoio durante o desenvolvimento, principalmente para esclarecer dúvidas pontuais relacionadas à sintaxe da linguagem, utilização de determinadas funções e formatação do código, alem de auxilio para a compreensão de certas lógicas na hora da implementação.

Prompts principais (liste, sem história): fiz perguntas sobre sintaxes específicas da linguagem, esclarecimentos sobre o funcionamento de determinadas estruturas e sugestões de formatação e organização do código.

O que validei manualmente e como: compilei e executei o código eu mesmo em todas as etapas, rodei o teste oficial do professor e os testes de caso-limite repetidamente no meu terminal WSL, conferi as saídas e os contadores internos da struct, e revisei a lógica de cada função para entender o porquê de cada decisão antes de aceitar o código.

8. Reflexão Final (5–8 linhas)

A implementação consistiu em traduzir as regras do enunciado em um pequeno conjunto de contadores (car_present, free_seats, waiting, to_board), protegidos por um único mutex e uma única condition variable,

sempre verificados dentro de while. A parte mais trabalhosa não foi o fluxo normal, e sim os casos de borda: carro sem assentos, carro sem ninguém esperando, e passageiros chegando depois que o embarque já havia começado. Testar manualmente esses casos, além do teste automático do professor, foi o que deu mais confiança na corretude da solução. Se fosse refazer, escreveria os testes de caso-limite antes de validar só o caminho feliz.

Reflexão:

9. Checklist Final de Entrega

Item	**Confirmado?**
Compilei do zero seguindo só o que está no relatório	OK
Rodei os testes e evidencias.log foi gerado	OK
Tenho prints obrigatórios	OK
Testei pelo menos um caso limite e um caso inválido	OK
Preenchi a seção de uso de IA	OK
Revisei o relatório e removi frases genéricas / vazias	OK

10. Se eu tivesse mais 2 horas

Separaria a condition variable única em duas (uma para “carro disponível”, outra para “embarque confirmado”), reduzindo wakeups desnecessários sem mudar a corretude. Também formalizaria os testes manuais em um script de regressão reutilizável e comentaria no código o significado de cada campo da struct e a invariante garantida por station_load_car ao retornar.

Regras de Integridade (não mudam)

• Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
• Contradição entre prints, log e relato = relatório perde credibilidade.
• Indício de evidência fabricada = entrega invalidada.